

AFRILANG Stdlib

std/c/graphflow

Français. Bibliothèque AFRILANG 'std/c/graphflow' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : maxFlow, minCutValue, flowMatrix, bottleneckCapacity, outFlowSum, inFlowSum, isFlowConserved.

```
'import "std/c/graphflow"' · 'use graphflow'
```

```
- 'maxFlow(adj list number, n number, source number, sink number)
→ number' - 'minCutValue(adj list number, n number, source number,
sink number) → number' - 'flowMatrix(adj list number, n number,
source number, sink number) → list number' - 'bottleneckCapacity(adj
list number, n number) → number' - 'outFlowSum(adj list number, n
number, node number) → English. AFRILANG library 'std/c/graphflow'
```

(tier complex, category complex). Exports 7 function(s): maxFlow, minCutValue, flowMatrix, bottleneckCapacity, outFlowSum, inFlowSum, isFlowConserved.

```
'import "std/c/graphflow"' · 'use graphflow'
```

```
- 'maxFlow(adj list number, n number, source number, sink number)
→ number' - 'minCutValue(adj list number, n number, source number,
sink number) → number' - 'flowMatrix(adj list number, n number,
source number, sink number) → list number' - 'bottleneckCapacity(adj
list number, n number) → number' - 'outFlowSum(adj list number, n
number, node number) → number' - 'i
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/graphflow' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : maxFlow, minCutValue, flowMatrix, bottleneckCapacity, outFlowSum, inFlowSum, isFlowConserved.

'import "std/c/graphflow"' · 'use graphflow'

```
- `maxFlow(adj list number, n number, source number, sink number) → number`
- `minCutValue(adj list number, n number, source number, sink number) → number`
- `flowMatrix(adj list number, n number, source number, sink number) → list`
- `bottleneckCapacity(adj list number, n number) → number`
- `outFlowSum(adj list number, n number, node number) → number`
- `inFlowSum(adj list number, n number, node number) → number`
- `isFlowConserved(adj list number, n number, source number, sink number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphflow"
use graphflow
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- maxFlow(adj list number, n number, source number, sink number) → number•
- minCutValue(adj list number, n number, source number, sink number) → number•
- flowMatrix(adj list number, n number, source number, sink number) → list•
- bottleneckCapacity(adj list number, n number) → number•
- outFlowSum(adj list number, n number, node number) → number•
- inFlowSum(adj list number, n number, node number) → number•
- isFlowConserved(adj list number, n number, source number, sink number) → bool

4. Exemples d'utilisation

```
import "std/c/graphflow"
use graphflow
```

```
create result = maxFlow(/* args */)
say result
```

```
create result = minCutValue(/* args */)
say result
```

```
create result = flowMatrix(/* args */)
say result
```

```
create result = bottleneckCapacity(/* args */)
say result
```

```
create result = outFlowSum(/* args */)
say result
```

```
create result = inFlowSum(/* args */)
say result
```

5. Bonnes pratiques

- Préférez `import "std/c/graphflow"` en tête de fichier.
- Lancez `afrilang check` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir /stdlib/ pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module graphflow

```
function maxFlow(adj list number, n number, source number, sink number) returns
  number
end
```

```
function minCutValue(adj list number, n number, source number, sink number) returns
  number
end
```

```
function flowMatrix(adj list number, n number, source number, sink number) returns
  list number
end
```

```
function bottleneckCapacity(adj list number, n number) returns number
end
```

```
function outFlowSum(adj list number, n number, node number) returns number
end
```

```
function inFlowSum(adj list number, n number, node number) returns number
end
```

```
function isFlowConserved(adj list number, n number, source number, sink number)
  returns bool
end
end
```

English

1. Overview

AFRILANG library 'std/c/graphflow' (tier complex, category complex). Exports 7 function(s): maxFlow, minCutValue, flowMatrix, bottleneckCapacity, outFlowSum, inFlowSum, isFlowConserved.

'import "std/c/graphflow"' · 'use graphflow'

```
- `maxFlow(adj list number, n number, source number, sink number) → number`
- `minCutValue(adj list number, n number, source number, sink number) → number`
- `flowMatrix(adj list number, n number, source number, sink number) → list`
- `bottleneckCapacity(adj list number, n number) → number`
- `outFlowSum(adj list number, n number, node number) → number`
- `inFlowSum(adj list number, n number, node number) → number`
- `isFlowConserved(adj list number, n number, source number, sink number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphflow"
use graphflow
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- maxFlow(adj list number, n number, source number, sink number) → number•
- minCutValue(adj list number, n number, source number, sink number) → number•
- flowMatrix(adj list number, n number, source number, sink number) → list•
- bottleneckCapacity(adj list number, n number) → number•
- outFlowSum(adj list number, n number, node number) → number•
- inFlowSum(adj list number, n number, node number) → number•
- isFlowConserved(adj list number, n number, source number, sink number) → bool

4. Usage examples

```
import "std/c/graphflow"
use graphflow
```

```
create result = maxFlow(/* args */)
say result
```

```
create result = minCutValue(/* args */)
say result
```

```
create result = flowMatrix(/* args */)
say result
```

```
create result = bottleneckCapacity(/* args */)
say result
```

```
create result = outFlowSum(/* args */)
say result
```

```
create result = inFlowSum(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/graphflow"`` at the top of your file.
- Run ``afriLang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module graphflow

```
function maxFlow(adj list number, n number, source number, sink number) returns
  number
end
```

```
function minCutValue(adj list number, n number, source number, sink number) returns
  number
end
```

```
function flowMatrix(adj list number, n number, source number, sink number) returns
  list number
end
```

```
function bottleneckCapacity(adj list number, n number) returns number
end
```

```
function outFlowSum(adj list number, n number, node number) returns number
end
```

```
function inFlowSum(adj list number, n number, node number) returns number
end
```

```
function isFlowConserved(adj list number, n number, source number, sink number)
  returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/graphflow"`
- Vérification : `afriLang check`
- Documentation : <https://afriLang.dev/stdlib/std/c/graphflow/>

Source (excerpt)

```
module graphflow

function maxFlow(adj list number, n number, source number, sink number) returns
    number
end

function minCutValue(adj list number, n number, source number, sink number) returns
    number
end

function flowMatrix(adj list number, n number, source number, sink number) returns
    list number
end

function bottleneckCapacity(adj list number, n number) returns number
end

function outFlowSum(adj list number, n number, node number) returns number
end

function inFlowSum(adj list number, n number, node number) returns number
end

function isFlowConserved(adj list number, n number, source number, sink number)
    returns bool
end
end
```